

ATIVIDADE 5

Documentação Técnica — JavaDoc

Sistema de Gestão Financeira Offline-First

Padrões GoF: Singleton · Proxy · Observer · Strategy · Command

Engenharia da Computação

1. Introdução

Este documento apresenta a documentação técnica JavaDoc completa para todos os protótipos implementados no Sistema de Gestão Financeira Offline-First. O sistema aplica cinco Padrões de Projeto GoF (Gang of Four) para garantir extensibilidade, desacoplamento e robustez em ambientes com conectividade instável. O padrão Singleton substitui o Factory Method nesta documentação, aplicando-se ao gerenciamento de instâncias únicas de recursos críticos.

A documentação segue o padrão JavaDoc, incluindo descrição de classes, descrição de métodos, parâmetros, retornos e observações relevantes sobre cada componente do sistema.

2. Padrão Singleton — Instância Única de Configuração do Sistema

Categoria: Criacional | Pacote sugerido: com.financeiro.config

O padrão Singleton garante que determinadas classes centrais do sistema possuam apenas uma instância durante todo o ciclo de vida da aplicação. No Sistema Financeiro Offline-First, esse padrão é aplicado em classes que gerenciam estado global compartilhado, como configurações de conexão, gerenciador de fila e o serviço de notificações, evitando instâncias duplicadas que causariam inconsistências de estado e desperdício de recursos.

<<class>>

ConfiguracaoSistema

Padrão: Singleton

Descrição

Singleton que centraliza todas as configurações do sistema (endpoints de API, credenciais de e-mail, intervalos de verificação de rede, limites de retry). Garante ponto único de acesso e modificação às propriedades da aplicação, evitando valores inconsistentes entre módulos.

```
/**
 * Configurações centrais do Sistema Financeiro — Singleton (GoF).
 *
 * <p>Mantém um único conjunto de propriedades de configuração acessível
 * por todos os módulos: URL da API de notificações, credenciais SMTP,
 * intervalo de polling de rede, número máximo de retries de sincronização
 * e caminho da planilha local de persistência offline.</p>
 *
 * <p>As configurações são carregadas na primeira chamada a
 * {@link #getInstancia()} a partir do arquivo {@code config.properties}
 * e podem ser sobrescritas em tempo de execução via
 * {@link #setPropriedade(String, String)}.</p>
 *
 * @author Equipe Financeiro
 * @version 1.0
 * @since Sprint 2
 */
public class ConfiguracaoSistema {

    private static volatile ConfiguracaoSistema instancia;
    private Properties props;
```

```

private ConfiguracaoSistema() {
    // carrega config.properties do classpath
}

/**
 * Retorna a instância única de configuração do sistema.
 *
 * <p>Na primeira chamada, carrega as propriedades do arquivo
 * {@code config.properties}. Chamadas subsequentes retornam
 * a mesma instância já inicializada.</p>
 *
 * @return instância singleton de {@code ConfiguracaoSistema}; nunca {@code
null}
 */
public static ConfiguracaoSistema getInstancia() { ... }

/**
 * Retorna o valor de uma propriedade de configuração.
 *
 * @param chave nome da propriedade (ex.: {@code "api.url"}); não deve ser nula
 * @return valor associado à chave, ou {@code null} se não encontrada
 * @throws IllegalArgumentException se {@code chave} for {@code null} ou vazia
 */
public String getPropriedade(String chave) { ... }

/**
 * Define ou atualiza o valor de uma propriedade em tempo de execução.
 *
 * <p><b>Observação:</b> alterações feitas em tempo de execução não são
 * persistidas no arquivo {@code config.properties}. Para persistência,
 * utilize {@link #salvarConfiguracoes()}.</p>
 *
 * @param chave nome da propriedade; não deve ser nula ou vazia
 * @param valor novo valor; {@code null} remove a propriedade
 * @throws IllegalArgumentException se {@code chave} for nula ou vazia
 */
public void setPropriedade(String chave, String valor) { ... }

/**
 * Persiste as configurações atuais no arquivo {@code config.properties}.
 *
 * @throws ConfiguracaoException se o arquivo não puder ser escrito
 * (ex.: sem permissão de escrita no diretório)
 */
public void salvarConfiguracoes() { ... }
}

```

3. Padrão Proxy — Controle de Acesso à Sincronização

Categoria: Estrutural | Pacote sugerido: `com.financeiro.sincronizacao`

```
<<interface>>
```

ISincronizacao

Padrão: Proxy

Descrição

Interface que abstrai o serviço de sincronização de dados. Representa o Subject no padrão Proxy, garantindo que tanto o serviço real quanto o proxy implementem o mesmo contrato, tornando a troca transparente para o código cliente.

```
/**
 * Contrato do serviço de sincronização de dados com a nuvem.
 *
 * <p>Representa o <b>Subject</b> no padrão Proxy (GoF). Tanto o
 * serviço real ({@link SincronizacaoReal}) quanto o proxy
 * ({@link ProxySincronizacao}) implementam esta interface,
 * garantindo transparência para o código cliente.</p>
 *
 * @author Equipe Financeiro
 * @version 1.0
 * @since Sprint 2
 */
public interface ISincronizacao {

    /**
     * Envia um payload de dados para o servidor remoto.
     *
     * @param payload dados serializados (JSON) a sincronizar;
     *             não deve ser nulo
     * @throws SincronizacaoException se a transmissão falhar e
     *             não houver mecanismo de retry configurado
     */
    void sincronizar(String payload);
}
```

<<class>>

ProxySincronizacao

Padrão: Proxy — RealProxy

Descrição

Proxy de proteção que intercepta chamadas ao serviço de sincronização e verifica se a conexão está ativa antes de delegar ao serviço real. Quando offline, bloqueia a chamada e direciona a operação para a fila de comandos pendentes (padrão Command). Também registra logs de tentativas de sincronização.

```
/**
 * Proxy de proteção para o serviço de sincronização remota.
 *
 * <p>Representa o <b>Proxy</b> no padrão Proxy (GoF). Verifica
 * o estado de conectividade – consultando {@link Conexao} –
 * antes de delegar ao {@link SincronizacaoReal}. Quando offline,
 * lança {@link SemConexaoException} para que a camada superior
 * encaminhe a operação à {@link FilaSincronizacao}.</p>
 *
 * <p>Também atua como {@link IObservadorConexao}: ao receber
 * evento de reconexão, processa automaticamente a fila pendente.</p>
 *
 * @author Equipe Financeiro
 * @version 1.0
 * @since Sprint 2
 * @see ISincronizacao
 * @see SincronizacaoReal
 * @see IObservadorConexao
 */
public class ProxySincronizacao implements ISincronizacao, IObservadorConexao {
```

```

/**
 * Verifica conectividade e delega ao serviço real se ONLINE.
 *
 * <p>Fluxo:</p>
 * <ol>
 *   <li>Consulta {@code Conexao.status} para verificar o estado atual.</li>
 *   <li>Se ONLINE: delega para {@link SincronizacaoReal#sincronizar}.</li>
 *   <li>Se OFFLINE: lança {@link SemConexaoException} com mensagem
 *       descritiva para que o chamador enfileire o comando.</li>
 * </ol>
 *
 * @param payload dados a sincronizar; não deve ser nulo
 * @throws SemConexaoException se o sistema estiver offline
 * @throws SincronizacaoException se o serviço real falhar
 */
@Override
public void sincronizar(String payload) { ... }

/**
 * Callback acionado pelo {@link Conexao} quando o status de rede muda.
 *
 * <p>Quando o novo status for {@code ONLINE}, dispara o processamento
 * da fila de comandos pendentes via {@link
FilaSincronizacao#processarFila}.</p>
 *
 * @param status novo estado de conectividade (ONLINE ou OFFLINE)
 */
@Override
public void atualizar(ConexaoStatus status) { ... }
}

```

<<class>>

SincronizacaoReal

Padrão: Proxy — RealSubject

```

/**
 * Implementação real do serviço de sincronização com a nuvem.
 *
 * <p>Representa o <b>RealSubject</b> no padrão Proxy (GoF). Realiza
 * a chamada efetiva à API remota para persistir ou atualizar dados.
 * Deve ser acessado exclusivamente por meio de {@link ProxySincronizacao},
 * nunca instanciado diretamente pelo código cliente.</p>
 *
 * @author Equipe Financeiro
 * @version 1.0
 * @since Sprint 2
 */
public class SincronizacaoReal implements ISincronizacao {

    /**
     * Envia dados para o servidor remoto via HTTP.
     *
     * @param payload JSON com os dados a persistir na nuvem;
     *             não deve ser nulo nem vazio
     * @throws SincronizacaoException se o servidor retornar
     *             código de erro HTTP ou a conexão for perdida
     *             durante a transmissão
     */
    @Override
    public void sincronizar(String payload) { ... }
}

```

```
}
```

4. Padrão Observer — Monitoramento de Conectividade

Categoria: Comportamental | Pacote sugerido: com.financeiro.conexao

<<interface>>

IObservadorConexao

Padrão: Observer

Descrição

Interface Observer que define o contrato para todos os componentes que precisam reagir a mudanças no estado de conectividade. Implementada por módulos como ProxySincronizacao (para processar fila ao reconectar) e pela interface do usuário (para exibir alertas).

```
/**
 * Interface Observer para eventos de mudança de conectividade.
 *
 * <p>Representa o <b>Observer</b> no padrão Observer (GoF). Qualquer
 * componente interessado em reagir a transições ONLINE/OFFLINE deve
 * implementar esta interface e se registrar em {@link Conexao}.</p>
 *
 * <p>Implementadores conhecidos:
 * {@link ProxySincronizacao}, {@code UIConexaoIndicador},
 * {@code SeletorEstrategiaSalvamento}.</p>
 *
 * @author Equipe Financeiro
 * @version 1.0
 * @since Sprint 2
 * @see Conexao
 */
public interface IObservadorConexao {

    /**
     * Notifica o observador sobre mudança no estado de conectividade.
     *
     * <p>Chamado pelo {@link Conexao} sempre que o status de rede
     * é alterado. A implementação deve ser rápida e não bloqueante;
     * operações pesadas devem ser despachadas para uma thread separada.</p>
     *
     * @param status novo estado da conexão
     *              ({@code ONLINE} ou {@code OFFLINE}); nunca {@code null}
     */
    void atualizar(ConexaoStatus status);
}
```

<<class>>

Conexao

Padrão: Observer — Subject

Descrição

Classe Subject do padrão Observer. Mantém o estado atual de conectividade da rede, registra observadores e os notifica automaticamente quando o status muda. É o ponto central de detecção de conectividade do sistema, sendo monitorada continuamente por um thread de verificação em background.

```
/**
 * Monitor de conectividade de rede – Subject do padrão Observer (GoF).
 *
 * <p>Mantém o estado atual da conexão ({@link ConexaoStatus}) e
 * gerencia a lista de {@link IObservadorConexao} registrados.
 * Ao detectar transição de estado, invoca {@link #notificarObservadores}
 * para propagar o evento a todos os interessados.</p>
 *
 * <p><b>Thread-safety:</b> os métodos {@link #adicionarObservador} e
 * {@link #notificarObservadores} devem ser sincronizados para evitar
 * {@code ConcurrentModificationException} em ambientes multi-thread.</p>
 *
 * @author Equipe Financeiro
 * @version 1.0
 * @since Sprint 2
 */
public class Conexao {

    /** Estado atual da conexão. */
    private ConexaoStatus status;

    /** Timestamp da última verificação bem-sucedida. */
    private LocalDateTime ultimaVerificacao;

    /**
     * Registra um observador para receber eventos de conectividade.
     *
     * <p>Se o observador já estiver registrado, a chamada é ignorada
     * silenciosamente (sem duplicar entradas na lista).</p>
     *
     * @param obs observador a registrar; não deve ser {@code null}
     * @throws IllegalArgumentException se {@code obs} for {@code null}
     */
    public void adicionarObservador(IObservadorConexao obs) { ... }

    /**
     * Notifica todos os observadores registrados sobre o estado atual.
     *
     * <p>Percorre a lista de observadores e chama
     * {@link IObservadorConexao#atualizar} em cada um, passando o
     * {@link #status} corrente. Deve ser invocado sempre que {@link #status}
     * for alterado.</p>
     *
     * <p><b>Observação:</b> falhas em um observador individual não devem
     * interromper a notificação dos demais – utilize try-catch por observador.</p>
     */
    public void notificarObservadores() { ... }
}
```

5. Padrão Strategy — Estratégia de Processamento de Transações

Categoria: Comportamental | Pacote sugerido: com.financeiro.transacao

```
<<interface>>
```

IEstrategiaProcessamento

Padrão: Strategy

Descrição

Interface Strategy que abstrai o algoritmo de processamento/salvamento de transações financeiras. Permite trocar o comportamento de salvamento em tempo de execução, dependendo do estado de conectividade, sem alterar a classe Transacao ou o código cliente.

```
/**
 * Contrato para estratégias de processamento de transações financeiras.
 *
 * <p>Representa a <b>Strategy</b> no padrão Strategy (GoF). Define o
 * algoritmo de salvamento/envio de uma {@link Transacao}, variando
 * conforme o estado de conectividade:
 * {@link ProcessamentoOnline} envia diretamente ao servidor;
 * {@link ProcessamentoOffline} persiste localmente e enfileira para sync.</p>
 *
 * @author Equipe Financeiro
 * @version 1.0
 * @since Sprint 2
 * @see Transacao
 * @see ProcessamentoOnline
 * @see ProcessamentoOffline
 */
public interface IEstrategiaProcessamento {

    /**
     * Processa (salva/envia) a transação financeira informada.
     *
     * <p>A implementação decide como e onde os dados serão persistidos:
     * remotamente (modo online) ou localmente com enfileiramento
     * (modo offline).</p>
     *
     * @param t transação a ser processada; não deve ser {@code null}
     * e deve ter todos os campos obrigatórios preenchidos
     * @throws ProcessamentoException se o processamento falhar e
     * não houver estratégia de fallback disponível
     */
    void processar(Transacao t);
}
```

<<class>>

Transacao

Padrão: Strategy — Context

Descrição

Entidade central do domínio financeiro e Context do padrão Strategy. Armazena os dados de uma transação financeira e delega o processamento à estratégia configurada, que pode ser trocada dinamicamente conforme o estado da rede.

```
/**
 * Entidade de domínio representando uma transação financeira.
 *
 * <p>Atua também como <b>Context</b> no padrão Strategy (GoF),
 * mantendo uma referência a {@link IEstrategiaProcessamento} e
 * delegando o processamento a ela via {@link #salvar()}.
 */
```



```

* A estratégia pode ser trocada em tempo de execução por
* {@link #setEstrategia(IEstrategiaProcessamento)}.</p>
*
* @author Equipe Financeiro
* @version 1.0
* @since Sprint 2
*/
public class Transacao {

    /** Identificador único da transação. */
    private String id;

    /** Valor monetário da transação. */
    private BigDecimal valor;

    /** Status atual (PENDENTE, SINCRONIZADO, ERRO). */
    private TransacaoStatus status;

    /** Estratégia de processamento em uso. */
    private IEstrategiaProcessamento estrategia;

    /**
     * Define (ou substitui) a estratégia de processamento ativa.
     *
     * <p>Permite trocar o comportamento em tempo de execução,
     * por exemplo quando o {@link IObservadorConexao} detecta
     * transição OFFLINE → ONLINE.</p>
     *
     * @param e nova estratégia de processamento; não deve ser {@code null}
     * @throws IllegalArgumentException se {@code e} for {@code null}
     */
    public void setEstrategia(IEstrategiaProcessamento e) { ... }

    /**
     * Processa esta transação usando a estratégia configurada.
     *
     * <p>Delega integralmente para {@link IEstrategiaProcessamento#processar}
     * passando {@code this} como argumento. Garante que uma estratégia
     * esteja definida antes de executar; caso contrário, lança
     * {@link IllegalStateException}.</p>
     *
     * @throws IllegalStateException se nenhuma estratégia foi configurada
     * @throws ProcessamentoException se a estratégia falhar ao processar
     */
    public void salvar() { ... }
}

```

```
<<class>>
```

ProcessamentoOnline

Padrão: Strategy — ConcreteStrategy

```

/**
 * Estratégia de processamento para o modo ONLINE.
 *
 * <p>Estratégia concreta do padrão Strategy (GoF). Envia a transação
 * diretamente ao servidor remoto via {@link ISincronizacao}. Deve ser
 * ativada pelo contexto quando a {@link Conexao} reportar status ONLINE.</p>
 *
 * @see IEstrategiaProcessamento
 * @see ProcessamentoOffline

```

```

*/
public class ProcessamentoOnline implements IEstrategiaProcessamento {

    /**
     * Envia a transação diretamente para o servidor remoto.
     *
     * @param t transação a processar; não deve ser {@code null}
     * @throws ProcessamentoException se a sincronização remota falhar
     */
    @Override
    public void processar(Transacao t) { ... }
}

```

<<class>>

ProcessamentoOffline

Padrão: Strategy — ConcreteStrategy

```

/**
 * Estratégia de processamento para o modo OFFLINE.
 *
 * <p>Estratégia concreta do padrão Strategy (GoF). Persiste a transação
 * na planilha local e cria um {@link IComandoSincronizacao} para
 * enfileiramento posterior. Garante que nenhum dado financeiro se perca
 * durante quedas de conectividade.</p>
 *
 * <p><b>Observação:</b> ao persistir localmente, define o status da
 * transação como {@code PENDENTE} e atribui um ID temporário até que
 * a sincronização confirme o ID definitivo do servidor.</p>
 *
 * @see IEstrategiaProcessamento
 * @see ProcessamentoOnline
 * @see FilaSincronizacao
 */
public class ProcessamentoOffline implements IEstrategiaProcessamento {

    /**
     * Persiste a transação localmente e a enfileira para sincronização.
     *
     * @param t transação a processar; não deve ser {@code null}
     * @throws ProcessamentoException se a persistência local falhar
     *         (ex.: planilha corrompida ou sem espaço em disco)
     */
    @Override
    public void processar(Transacao t) { ... }
}

```

6. Padrão Command — Fila de Sincronização

Categoria: Comportamental | Pacote sugerido: com.financeiro.fila

<<interface>>

IComandoSincronizacao

Padrão: Command

Descrição

Interface Command que define o contrato para todas as operações pendentes de sincronização. Cada operação do usuário realizada em modo offline é encapsulada como um objeto Command, permitindo enfileiramento, reexecução e auditoria das operações pendentes.

```
/**
 * Contrato para comandos de sincronização pendentes.
 *
 * <p>Representa o <b>Command</b> no padrão Command (GoF). Cada operação
 * realizada offline (criar/alterar transação, enviar notificação) é
 * encapsulada como um objeto que implementa esta interface, tornando-a
 * enfileirável, serializável e reexecutável.</p>
 *
 * <p><b>Idempotência:</b> implementações devem ser projetadas para
 * execução segura mais de uma vez (ex.: usar upsert no servidor),
 * pois retry pode reexecutar o mesmo comando em caso de falha parcial.</p>
 *
 * @author Equipe Financeiro
 * @version 1.0
 * @since Sprint 2
 * @see FilaSincronizacao
 * @see SincronizarTransacaoCommand
 */
public interface IComandoSincronizacao {

    /**
     * Executa a operação de sincronização encapsulada neste comando.
     *
     * <p>Deve realizar a comunicação com o serviço remoto e, em caso
     * de sucesso, marcar o estado interno como CONCLUÍDO para evitar
     * reprocessamento. Em caso de falha recuperável, deve lançar
     * {@link SincronizacaoException} para que a fila tente novamente.
     * Falhas irreversíveis devem lançar {@link ComandoInvalidoException}.</p>
     *
     * @throws SincronizacaoException se a execução falhar de forma
     * recuperável (ex.: timeout de rede)
     * @throws ComandoInvalidoException se os dados do comando forem
     * inválidos ou o servidor rejeitar
     * permanentemente a operação
     */
    void executar();
}
```

<<class>>

FilaSincronizacao

Padrão: Command — Invoker

Descrição

Invoker do padrão Command. Gerencia a fila de operações pendentes de sincronização, mantendo os comandos em ordem de chegada (FIFO) e os processando quando a conexão é restabelecida. Funciona como a Outbox do sistema, garantindo resiliência total contra quedas de conectividade.

```
/**
 * Fila de comandos pendentes de sincronização — Invoker do padrão Command (GoF).
 *
 * <p>Armazena instâncias de {@link IComandoSincronizacao} em ordem FIFO
 * e as processa quando o sistema estiver ONLINE. Funciona como a
 * <em>Outbox</em> do sistema, garantindo que nenhuma operação offline
```

```

* se perca durante quedas de conectividade.</p>
*
* <p><b>Persistência:</b> recomenda-se persistir os comandos da fila
* localmente (planilha ou banco embutido) para sobreviver a reinicializações
* da aplicação.</p>
*
* @author Equipe Financeiro
* @version 1.0
* @since Sprint 2
*/
public class FilaSincronizacao {

    /** Lista interna de comandos aguardando execução. */
    private List<IComandoSincronizacao> comandos;

    /**
     * Adiciona um novo comando ao final da fila.
     *
     * <p>Deve ser chamado sempre que uma operação for realizada
     * offline (ex.: {@link ProcessamentoOffline#processar}).
     * A fila aceita qualquer implementação de {@link IComandoSincronizacao}.</p>
     *
     * @param cmd comando a enfileirar; não deve ser {@code null}
     * @throws IllegalArgumentException se {@code cmd} for {@code null}
     */
    public void enfileirar(IComandoSincronizacao cmd) { ... }

    /**
     * Processa todos os comandos pendentes na ordem em que foram enfileirados.
     *
     * <p>Para cada comando:</p>
     * <ol>
     * <li>Chama {@link IComandoSincronizacao#executar()}.</li>
     * <li>Em caso de sucesso: remove da fila.</li>
     * <li>Em caso de {@link SincronizacaoException}: mantém na fila
     * para nova tentativa (máximo configurável de retries).</li>
     * <li>Em caso de {@link ComandoInvalidoException}: descarta o
     * comando e registra no log de erros.</li>
     * </ol>
     *
     * <p><b>Observação:</b> deve ser invocado pelo {@link ProxySincronizacao}
     * ao receber evento ONLINE via {@link IObservadorConexao#atualizar}.</p>
     *
     * @throws IllegalStateException se chamado enquanto o sistema
     * ainda estiver no estado OFFLINE
     */
    public void processarFila() { ... }
}

```

```
<<class>>
```

SincronizarTransacaoCommand

Padrão: Command — ConcreteCommand

Descrição

Comando concreto que encapsula a operação de sincronização de uma transação financeira específica. Mantém referência à transação e ao receptor (ISincronizacao), chamando o serviço no momento da execução.

```
/**
```

```

* Comando concreto para sincronização de uma {@link Transacao} pendente.
*
* <p>Representa um <b>ConcreteCommand</b> no padrão Command (GoF).
* Encapsula a transação financeira que deve ser sincronizada e o
* receptor ({@link ISincronizacao}) responsável por executar a operação,
* permitindo enfileiramento e execução posterior.</p>
*
* @author Equipe Financeiro
* @version 1.0
* @since Sprint 2
*/
public class SincronizarTransacaoCommand implements IComandoSincronizacao {

    /** Transação a ser sincronizada. */
    private Transacao transacao;

    /** Receptor que executará a sincronização remota. */
    private ISincronizacao receptor;

    /**
     * Executa a sincronização da transação encapsulada.
     *
     * <p>Serializa {@link #transacao} para JSON e delega a chamada
     * ao {@link #receptor}. Ao concluir com sucesso, atualiza o
     * status da transação para {@code SINCRONIZADO}.
     * Em caso de falha, mantém o status {@code PENDENTE} para
     * que a fila possa retentar.</p>
     *
     * @throws SincronizacaoException se a transmissão remota falhar
     * @throws ComandoInvalidoException se a transação for inválida
     * ou rejeitada permanentemente pelo servidor
     */
    @Override
    public void executar() { ... }
}

```

7. Exceções do Sistema

As exceções abaixo devem ser criadas como classes checked ou unchecked conforme a política de tratamento de erros do projeto:

```

/**
 * Exceção lançada quando uma notificação não pode ser entregue.
 * Usado por: NotificacaoAPI, NotificacaoEmail.
 */
public class NotificacaoException extends RuntimeException { ... }

/**
 * Exceção lançada quando a sincronização remota falha de forma recuperável.
 * Indica que o comando deve permanecer na fila para retry.
 */
public class SincronizacaoException extends RuntimeException { ... }

/**
 * Exceção lançada quando o sistema tenta sincronizar estando offline.
 * Sinaliza ao chamador para enfileirar a operação via Command.
 */
public class SemConexaoException extends RuntimeException { ... }

/**
 * Exceção lançada quando um comando possui dados inválidos e
 * não deve ser reexecutado (falha irrecuperável).
 */

```

```

*/
public class ComandoInvalidoException extends RuntimeException { ... }

/**
 * Exceção geral de processamento de transação financeira.
 */
public class ProcessamentoException extends RuntimeException { ... }

```

8. Sumário Geral de Classes e Métodos

Classe / Interface	Padrão GoF	Responsabilidade
ConfiguracaoSistema	Singleton	Instância única das configurações da app
ISincronizacao	Proxy	Contrato do serviço de sync (Subject)
ProxySincronizacao	Proxy	Valida conectividade antes de sincronizar
SincronizacaoReal	Proxy	Sincroniza com a nuvem (RealSubject)
IObservadorConexao	Observer	Contrato de observação de rede (Observer)
Conexao	Observer	Detecta e notifica mudanças de rede (Subject)
IEstrategiaProcessamento	Strategy	Contrato de processamento (Strategy)
Transacao	Strategy	Entidade financeira e Context do padrão
ProcessamentoOnline	Strategy	Salva diretamente no servidor (modo online)
ProcessamentoOffline	Strategy	Salva localmente e enfileira (modo offline)
IComandoSincronizacao	Command	Contrato de operação enfileirável (Command)
FilaSincronizacao	Command	Gerencia fila FIFO de comandos (Invoker)
SincronizarTransacaoCommand	Command	Encapsula sync de uma transação (ConcreteCmd)